

Project Part-2

Leo Lu

2024-12-04

Contents

1	Brief Overview of the Data	1
2	Random Sample	3
3	Multiple Linear Regression	3
4	Checking Multicollinearity	4
4.1	Variance Inflation Factor	4
4.2	Using Condition Index	4
5	Variable Selection of Regression Model	5
5.1	Backward Stepwise Elemination	5
5.2	Forward Stepwise Elemination	6
6	Cross Validation	7

1 Brief Overview of the Data

```
# Data Loading
diamonds <- read_csv("Diamonds Prices2022.csv")
```

```
## New names:
## Rows: 53943 Columns: 11
## -- Column specification
```

```
## ----- Delimiter: "," chr
## (3): cut, color, clarity dbl (8): ...1, carat, depth, table, price, x, y, z
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...1`
```

Data Preview

```
head(diamonds)
```

Adjusting Data

```
diamonds = diamonds[,-1]
str(diamonds)
```

```
## tibble [53,943 x 10] (S3: tbl_df/tbl/data.frame)
## $ carat : num [1:53943] 0.23 0.21 0.23 0.29 0.31 0.24 0.24 0.26 0.22 0.23 ...
## $ cut : chr [1:53943] "Ideal" "Premium" "Good" "Premium" ...
## $ color : chr [1:53943] "E" "E" "E" "I" ...
## $ clarity: chr [1:53943] "SI2" "SI1" "VS1" "VS2" ...
## $ depth : num [1:53943] 61.5 59.8 56.9 62.4 63.3 62.8 62.3 61.9 65.1 59.4 ...
## $ table : num [1:53943] 55 61 65 58 58 57 57 55 61 61 ...
## $ price : num [1:53943] 326 326 327 334 335 336 336 337 337 338 ...
## $ x : num [1:53943] 3.95 3.89 4.05 4.2 4.34 3.94 3.95 4.07 3.87 4 ...
## $ y : num [1:53943] 3.98 3.84 4.07 4.23 4.35 3.96 3.98 4.11 3.78 4.05 ...
## $ z : num [1:53943] 2.43 2.31 2.31 2.63 2.75 2.48 2.47 2.53 2.49 2.39 ...
```

```
diamonds$cut = factor(diamonds$cut,
  levels = c('Fair', 'Good', 'Very Good', 'Premium', 'Ideal'))
diamonds$color = factor(diamonds$color,
  levels = c('D', 'E', 'F', 'G', 'H', 'I', 'J'))
diamonds$clarity = factor(diamonds$clarity,
  levels = c('I1', 'SI2', 'SI1', 'VS2', 'VS1', 'VVS2', 'VVS1', 'IF'))
str(diamonds)
```

```
## tibble [53,943 x 10] (S3: tbl_df/tbl/data.frame)
## $ carat : num [1:53943] 0.23 0.21 0.23 0.29 0.31 0.24 0.24 0.26 0.22 0.23 ...
## $ cut : Factor w/ 5 levels "Fair","Good",...: 5 4 2 4 2 3 3 3 1 3 ...
## $ color : Factor w/ 7 levels "D","E","F","G",...: 2 2 2 6 7 7 6 5 2 5 ...
## $ clarity: Factor w/ 8 levels "I1","SI2","SI1",...: 2 3 5 4 2 6 7 3 4 5 ...
## $ depth : num [1:53943] 61.5 59.8 56.9 62.4 63.3 62.8 62.3 61.9 65.1 59.4 ...
## $ table : num [1:53943] 55 61 65 58 58 57 57 55 61 61 ...
## $ price : num [1:53943] 326 326 327 334 335 336 336 337 337 338 ...
## $ x : num [1:53943] 3.95 3.89 4.05 4.2 4.34 3.94 3.95 4.07 3.87 4 ...
## $ y : num [1:53943] 3.98 3.84 4.07 4.23 4.35 3.96 3.98 4.11 3.78 4.05 ...
## $ z : num [1:53943] 2.43 2.31 2.31 2.63 2.75 2.48 2.47 2.53 2.49 2.39 ...
```

2 Random Sample

```
set.seed(111)
index = sample(1:nrow(diamonds), size = 500, replace = F)
diamonds_sample = diamonds[index, ]
```

3 Multiple Linear Regression

```
fit <- lm(price ~ carat + depth + cut + color + table, data = diamonds_sample)
summary(fit)
```

```
##
## Call:
## lm(formula = price ~ carat + depth + cut + color + table, data = diamonds_sample)
##
## Residuals:
```

	Min	1Q	Median	3Q	Max
	-6709.6	-707.6	-125.4	480.5	6814.6

```
##
## Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	7650.43	5443.33	1.405	0.160520
carat	7999.08	151.82	52.690	< 2e-16 ***
depth	-117.76	57.43	-2.050	0.040862 *
cutGood	1201.65	470.53	2.554	0.010958 *
cutVery Good	1456.63	472.81	3.081	0.002182 **
cutPremium	1811.76	477.30	3.796	0.000166 ***
cutIdeal	1715.61	499.07	3.438	0.000637 ***
colorE	311.80	250.94	1.243	0.214643
colorF	221.67	251.39	0.882	0.378325
colorG	215.49	252.19	0.854	0.393258
colorH	-441.70	268.86	-1.643	0.101057
colorI	-613.16	308.79	-1.986	0.047630 *
colorJ	-1639.18	343.91	-4.766	2.48e-06 ***
table	-75.90	41.92	-1.811	0.070830 .

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1489 on 486 degrees of freedom
## Multiple R-squared:  0.8589, Adjusted R-squared:  0.8551
## F-statistic: 227.6 on 13 and 486 DF,  p-value: < 2.2e-16
```

4 Checking Multicollinearity

4.1 Variance Inflation Factor

```
X = model.matrix(fit)[-1]
R = cor(X)
diag(solve(R))
```

```
##      carat      depth    cutGood cutVery Good  cutPremium  cutIdeal
##  1.161937  1.785249  4.806987   8.299989  10.025048  13.330022
##    colorE    colorF    colorG    colorH    colorI    colorJ
##  2.219249  2.261790  2.276219  2.031330  1.795250  1.503473
##      table
##  2.069893
```

```
W <- scale(X)
W <- W / sqrt(nrow(X) - 1)
diag(solve(t(W)%*%W))
```

```
##      carat      depth    cutGood cutVery Good  cutPremium  cutIdeal
##  1.161937  1.785249  4.806987   8.299989  10.025048  13.330022
##    colorE    colorF    colorG    colorH    colorI    colorJ
##  2.219249  2.261790  2.276219  2.031330  1.795250  1.503473
##      table
##  2.069893
```

```
faraway::vif(fit)
```

```
##      carat      depth    cutGood cutVery Good  cutPremium  cutIdeal
##  1.161937  1.785249  4.806987   8.299989  10.025048  13.330022
##    colorE    colorF    colorG    colorH    colorI    colorJ
##  2.219249  2.261790  2.276219  2.031330  1.795250  1.503473
##      table
##  2.069893
```

These three approaches will give us three same answers where we can observed that there are two VIFs bigger than 10. However, both of cutPremium and cutIdeal are dummy variables from categorical variables, so it is not appropriate to exclude that covariate.

4.2 Using Condition Index

```
eigenval = eigen(R)$values
sqrt(max(eigenval)/eigenval)
```

```
## [1] 1.000000 1.201916 1.238705 1.318986 1.325970 1.345158 1.379190 1.412707
## [9] 1.600331 1.753485 2.409204 4.052389 8.628496
```

As we can see for the results that there are no one number bigger than 30 or even bigger than 10, so there is no severe multicollinearity.

5 Variable Selection of Regression Model

5.1 Backward Stepwise Elimination

```
library(MASS)
```

```
##
```

```
## 'MASS'
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
## select
```

```
fit1 <- lm(price ~ carat + depth + cut + color + table, data = diamonds)
fit1_step <- stepAIC(fit1, direction="both", k = 2, trace = 0)
summary(fit1_step)
```

```
##
```

```
## Call:
```

```
## lm(formula = price ~ carat + depth + cut + color + table, data = diamonds)
```

```
##
```

```
## Residuals:
```

```
##      Min       1Q   Median       3Q      Max
## -17302.6  -751.0   -83.2    543.8  12163.4
```

```
##
```

```
## Coefficients:
```

```
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2228.841    456.687   4.880 1.06e-06 ***
## carat       8200.111     13.959  587.429 < 2e-16 ***
## depth      -57.719      5.032 -11.471 < 2e-16 ***
```

```
## cutGood      1019.639      42.104  24.217 < 2e-16 ***
## cutVery Good 1350.844      40.261  33.552 < 2e-16 ***
## cutPremium   1272.963      40.353  31.546 < 2e-16 ***
## cutIdeal     1558.095      41.775  37.297 < 2e-16 ***
## colorE       -90.792      22.596  -4.018 5.88e-05 ***
## colorF       -74.861      22.744  -3.291 0.000997 ***
## colorG      -104.764      22.043  -4.753 2.01e-06 ***
## colorH      -728.628      23.678 -30.772 < 2e-16 ***
## colorI     -1070.294      26.549 -40.314 < 2e-16 ***
## colorJ     -1901.785      32.830 -57.928 < 2e-16 ***
## table        -39.125       3.680 -10.631 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1430 on 53929 degrees of freedom
## Multiple R-squared:  0.8716, Adjusted R-squared:  0.8715
## F-statistic: 2.815e+04 on 13 and 53929 DF,  p-value: < 2.2e-16
```

5.2 Forward Stepwise Elimination

```
fit2 = lm(price ~ 1, data = diamonds)
fit2_step <- stepAIC(fit2, direction="both",
                    scope=list(upper=formula(fit1)), k = 2, trace = 0)
summary(fit2_step)
```

```
##
## Call:
## lm(formula = price ~ carat + color + cut + depth + table, data = diamonds)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -17302.6   -751.0    -83.2    543.8   12163.4
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2228.841    456.687   4.880 1.06e-06 ***
## carat         8200.111     13.959  587.429 < 2e-16 ***
## colorE        -90.792     22.596  -4.018 5.88e-05 ***
## colorF        -74.861     22.744  -3.291 0.000997 ***
## colorG       -104.764     22.043  -4.753 2.01e-06 ***
## colorH       -728.628     23.678 -30.772 < 2e-16 ***
## colorI     -1070.294     26.549 -40.314 < 2e-16 ***
```

```
## colorJ          -1901.785      32.830 -57.928 < 2e-16 ***
## cutGood         1019.639      42.104  24.217 < 2e-16 ***
## cutVery Good   1350.844      40.261  33.552 < 2e-16 ***
## cutPremium     1272.963      40.353  31.546 < 2e-16 ***
## cutIdeal       1558.095      41.775  37.297 < 2e-16 ***
## depth          -57.719       5.032 -11.471 < 2e-16 ***
## table          -39.125       3.680 -10.631 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1430 on 53929 degrees of freedom
## Multiple R-squared:  0.8716, Adjusted R-squared:  0.8715
## F-statistic: 2.815e+04 on 13 and 53929 DF,  p-value: < 2.2e-16
```

6 Cross Validation

```
KFoldCV_lm = function(data, response, formula, k,
                      seed = FALSE, seed_num, shuffle = FALSE) {
  if (seed == TRUE) {
    set.seed(seed_num)
  }
  if(shuffle == TRUE) {
    shuffle_index = sample(1:nrow(data), replace = FALSE)
    data = data[shuffle_index,]
    response = response[shuffle_index]
  }

  folds <- cut(seq(1,nrow(data)),breaks=k,labels=FALSE)
  mse = numeric()
  for(i in 1:k){
    testIndexes <- which(folds == i, arr.ind=TRUE)
    testData <- data[testIndexes, ]
    trainData <- data[-testIndexes, ]
    fit.train = lm(formula, data = trainData)
    mse[i] =
      (1/length(testIndexes))*sum((response[testIndexes] -
                                   predict(fit.train, newdata = testData))^2)
  }
  rmse = sqrt(mse)
  cv_k_mse = sum(mse)/k
  cv_k_rmse = sum(rmse)/k
  return(list(CV_MSE = cv_k_mse, CV_RMSE = cv_k_rmse))
}
```

```
}
```

```
KFoldCV_lm(response = diamonds$price,  
            formula =price ~ carat + depth + cut + color + table,  
            data = diamonds,  
            k = 10, shuffle = TRUE, seed = TRUE, seed_num = 1)
```

```
## $CV_MSE  
## [1] 2045164  
##  
## $CV_RMSE  
## [1] 1429.692
```

```
KFoldCV_lm(response = diamonds$price,  
            formula =price ~ carat + cut + color,  
            data = diamonds,  
            k = 10, shuffle = TRUE, seed = TRUE, seed_num = 1)
```

```
## $CV_MSE  
## [1] 2051609  
##  
## $CV_RMSE  
## [1] 1431.963
```